UNIVERSITÀ DEGLI STUDI DI FIRENZE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Applicativo Java EscapeManager

Autore:
Adriano Luca Castaldo

N° Matricola:
7138545

Corso principale:
Ingegneria del software

Docente corso:
Enrico Vicario

Anno Accademico 2025/2026

Indice

1 Introduzione **2**
1.1 Statement 2

2 Analisi dei Requisiti **3**
2.1 Use Case Diagram (High-Level) 3
2.2 Use Case Diagram (Dettagliato) 4
2.3 Use Case Templates 5
2.3.1 UC1 - Prenotazione Stanza 6
2.3.2 UC2 - Iscrizione Lista d'Attesa (Extend) 6
2.3.3 UC3 - Controllo Sessione di Gioco 7

3 Progettazione Architetturale e di Dominio **9**
3.1 Domain Model (Class Diagram) 9
3.1.1 Scelte Architetturali e Design Pattern 10
3.2 Progettazione del Database 11
3.2.1 Diagramma Entity-Relationship 11
3.2.2 Strategie di Mapping e Modello Logico 12

4 Implementazione e Architettura del Codice **14**
4.1 Architettura dei Package 14
4.2 Livello di Persistenza (Pattern DAO) 15
4.3 Implementazione dei Design Pattern 15
4.3.1 State Pattern 15
4.3.2 Strategy Pattern 16
4.3.3 Observer Pattern 16

5 Collaudo e Testing **18**
5.1 Testing Strutturale (White-Box) 18
5.2 Testing di Integrazione (DAO e Database) 19
5.3 Testing Funzionale (Black-Box e Tracciabilità) 19
5.4 Conclusioni 20

Capitolo 1

Introduzione

1.1 Statement

Il progetto **EscapeManager** modella un sistema software centralizzato per la gestione operativa e commerciale di un franchise di Escape Room. L'obiettivo principale dell'applicativo è fornire un'infrastruttura unificata che permetta ai clienti di esplorare le stanze tematiche disponibili ed effettuare prenotazioni e, allo stesso tempo, mette a disposizione del personale strumenti avanzati per il controllo delle sessioni di gioco e la gestione commerciale.

Le sedi del franchise ospitano diverse stanze fisiche (Room), ciascuna caratterizzata da un tema specifico, un livello di difficoltà, una capienza massima di giocatori e un ciclo di vita operativo ben definito (es. *Disponibile, In Corso, In Manutenzione, In Pulizia*).

Il sistema è progettato per supportare tre tipologie principali di utenti:

- **Cliente:** può visualizzare il catalogo globale delle stanze filtrando per sede o difficoltà, prenotare una sessione di gioco per la propria squadra in uno slot temporale libero, e iscriversi a un sistema di avvisi per ricevere notifiche in tempo reale qualora si liberasse un orario precedentemente al completo.

- **Game Master (Operatore):** assegnato a una specifica filiale, gestisce il flusso operativo quotidiano. Può monitorare le prenotazioni del giorno, avviare o terminare le sessioni di gioco, aggiornare dinamicamente lo stato delle stanze (es. bloccando una stanza per un guasto tecnico a un enigma) e registrare l'esito della partita (vittoria/sconfitta e tempo di fuga).

- **Admin (Amministratore):** possiede una visione d'insieme sull'intero franchise. Ha i privilegi per aggiungere nuove filiali, inserire nuovi temi per le stanze e configurare politiche di prezzo dinamiche (es. tariffe maggiorate nei giorni festivi o sconti last-minute per riempire gli slot invenduti).

Per garantire manutenibilità e scalabilità, l'architettura fa uso di principi di Object-Oriented Design e di Design Pattern strutturali e comportamentali per risolvere problemi specifici del dominio, come la gestione delle variazioni di stato delle stanze, l'applicazione delle tariffe e il sistema di notifiche. La persistenza dei dati, fondamentale per evitare sovrapposizioni nelle prenotazioni e mantenere lo storico delle partite, è gestita tramite un database relazionale connesso al sistema.

Capitolo 2

Analisi dei Requisiti

Per definire i requisiti funzionali del sistema in ottica Object-Oriented, si è proceduto all'identificazione degli attori principali e dei relativi casi d'uso. La modellazione è stata divisa in due livelli di astrazione: un diagramma di contesto ad alto livello per identificare i macro-obiettivi (User Goals) e un diagramma dettagliato per l'analisi tecnica.

Il sistema prevede le seguenti figure:

- **Cliente:** Utente finale che interagisce col sistema per consultare il catalogo e prenotare le esperienze.
- **Game Master:** Personale di filiale deputato al controllo operativo delle stanze e delle sessioni.
- **Admin:** Amministratore di sistema incaricato di definire il catalogo e le logiche commerciali.

2.1 Use Case Diagram (High-Level)

La Figura 2.1 illustra il diagramma di contesto. Questo livello omette le dipendenze strutturali per evidenziare esclusivamente il macro-scopo di ciascun attore all'interno dell'ecosistema EscapeManager.

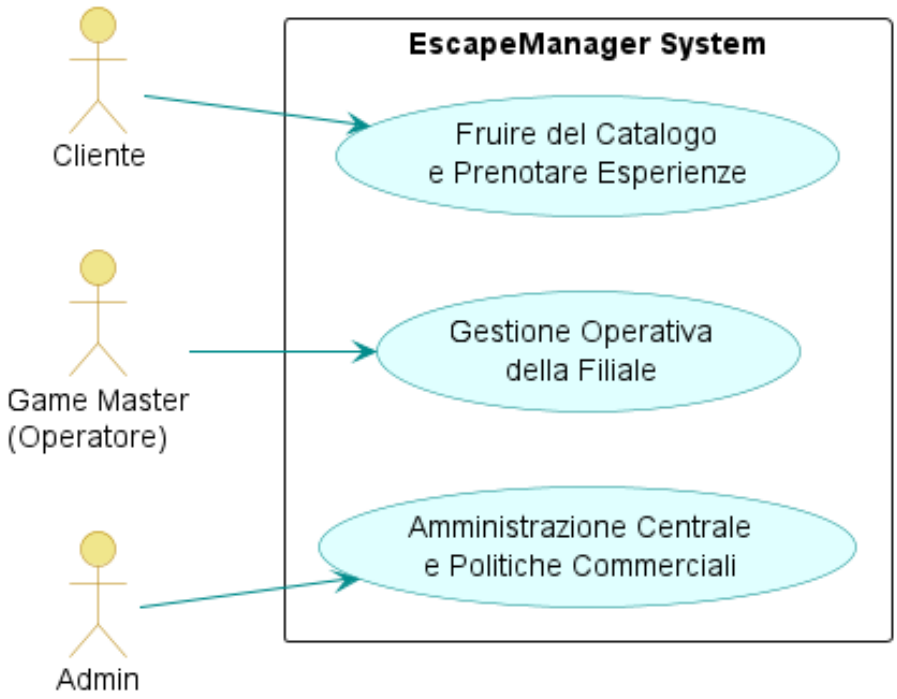

Figura 2.1: Diagramma di Contesto (Macro-obiettivi degli attori)

2.2 Use Case Diagram (Dettagliato)

La Figura 2.2 espande i macro-obiettivi nei casi d'uso effettivi del sistema. In questa vista vengono esplicitate le relazioni di inclusione («`include`») necessarie per la sicurezza e l'autenticazione, e le relazioni di estensione («`extend`») che modellano flussi alternativi complessi, come la gestione delle liste d'attesa.

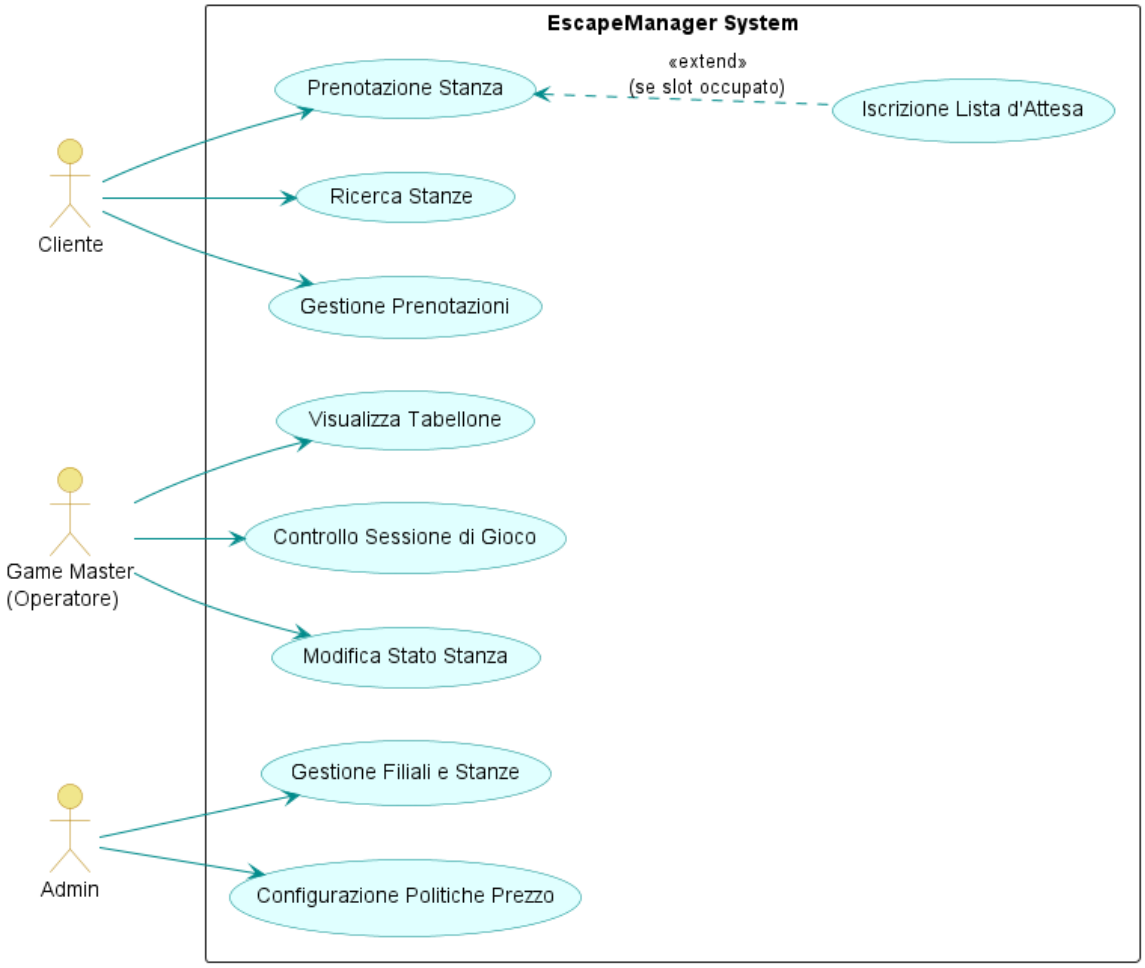

Figura 2.2: Diagramma dei Casi d'Uso Dettagliato

2.3 Use Case Templates

Di seguito vengono dettagliati i template per i casi d'uso primari del sistema, adottando un formato strutturato che esplicita le pre-condizioni, le post-condizioni, il flusso base (Main Success Scenario) e le possibili deviazioni (Alternative Flows). Questa formalizzazione costituisce il contratto per lo sviluppo e la base per la successiva stesura dei test funzionali.

2.3.1 UC1 - Prenotazione Stanza

Nome Use Case	UC1 - Prenotazione Stanza
Attore Principale	Cliente
Scopo	Permettere al cliente di riservare una stanza in uno specifico slot temporale, calcolando la tariffa appropriata.
Pre-condizioni	Il Cliente è autenticato nel sistema. Il sistema ha caricato il catalogo delle stanze.
Post-condizioni	Una nuova Prenotazione viene registrata nel sistema con stato "Confermata". Lo slot orario della stanza risulta occupato.
Flusso Base (Basic Flow)	
1. Il Cliente seleziona una Stanza dal catalogo e richiede di visualizzare le disponibilità. 2. Il Sistema mostra il calendario con le date e gli slot orari disponibili per la Stanza scelta. 3. Il Cliente seleziona una data, uno slot orario libero e inserisce il numero di giocatori. 4. Il Sistema calcola il prezzo totale basandosi sulla strategia di prezzo corrente (es. base, weekend) e mostra il riepilogo. 5. Il Cliente conferma la prenotazione. 6. Il Sistema registra la prenotazione, aggiorna lo stato dello slot orario e mostra un messaggio di successo.	
Flussi Alternativi (Alternative Flows)	
3a. Slot orario non più disponibile (Race condition): 3a.1 Se nel momento della conferma lo slot è stato appena prenotato da un altro utente, il Sistema notifica l'errore. 3a.2 Il Sistema propone al Cliente di tornare al passo 2 o di iscriversi alla Lista d'Attesa (UC2 - *Extend*). **4a. Numero di giocatori non valido:** 4a.1 Se il numero di giocatori inserito supera la capienza massima (o è inferiore alla minima) della Stanza, il Sistema segnala l'errore. 4a.2 Il Cliente è riportato al passo 3 per correggere il dato.	

Tabella 2.1: Template del Caso d'Uso: Prenotazione Stanza

2.3.2 UC2 - Iscrizione Lista d'Attesa (Extend)

Questo caso d'uso rappresenta un'estensione del normale flusso di prenotazione (UC1) e si attiva esclusivamente quando il Cliente tenta di prenotare uno slot orario che ha già raggiunto

la capienza massima.

Nome Use Case	UC2 - Iscrizione Lista d'Attesa
Attore Principale	Cliente
Scopo	Iscrivere l'utente a un sistema di notifiche per uno slot occupato, avvisandolo in caso di disdetta.
Pre-condizioni	Il Cliente è autenticato. Lo slot orario desiderato per la Stanza risulta "Occupato" (condizione di estensione dal passo 3a dell'UC1).
Post-condizioni	Il Cliente è inserito nella lista d'attesa per quello specifico slot. Nessuna prenotazione è ancora confermata.
Flusso Base (Basic Flow)	
1. Il Cliente seleziona l'opzione "Avvisami se si libera". 2. Il Sistema registra l'iscrizione del Cliente come "Osservatore" per quello specifico slot e Stanza. 3. Il Sistema mostra un messaggio di conferma e riporta l'utente alla dashboard principale.	
Flusso Asincrono (Asynchronous Flow)	
Notifica di disponibilità: A.1 Un altro utente cancella la sua prenotazione per quello slot. A.2 Il Sistema rileva il cambio di stato dello slot (da Occupato a Libero). A.3 Il Sistema notifica automaticamente e in tempo reale tutti i Clienti iscritti alla lista d'attesa per quello slot. A.4 La lista d'attesa per quello slot viene svuotata o i clienti vengono rimossi man mano che uno di loro effettua la prenotazione.	

Tabella 2.2: Template del Caso d'Uso: Iscrizione Lista d'Attesa

2.3.3 UC3 - Controllo Sessione di Gioco

Questo caso d'uso è il cuore operativo della filiale e modella il modo in cui il Game Master interagisce con il ciclo di vita della Stanza fisica.

Nome Use Case	UC3 - Controllo Sessione di Gioco
Attore Principale	Game Master
Scopo	Gestire lo svolgimento di una partita, alterando lo stato della stanza e registrando l'esito finale.
Pre-condizioni	Il Game Master è autenticato. Esiste una prenotazione per l'orario corrente. La Stanza è in stato "Disponibile".
Post-condizioni	La partita è conclusa, l'esito è salvato a database e la Stanza passa in stato "In Pulizia".

Flusso Base (Basic Flow)

1. Il Game Master seleziona la prenotazione in partenza dal tabellone giornaliero.
2. Il Game Master preme il comando "Avvia Sessione".
3. Il Sistema transita lo stato della Stanza da "Disponibile" a "In Corso" e fa partire il timer interno.
4. I giocatori terminano la partita (risolvono l'ultimo enigma o scade il tempo massimo).
5. Il Game Master preme il comando "Termina Sessione" e inserisce l'esito (Vittoria/Sconfitta) e il tempo impiegato.
6. Il Sistema salva le statistiche nel database e transita lo stato della Stanza da "In Corso" a "In Pulizia".

Flussi Alternativi (Alternative Flows)

2a. Stanza non disponibile (Manutenzione):

2a.1 Se la Stanza si trova in stato "In Manutenzione", il Sistema blocca l'avvio della sessione e mostra un avviso.

2a.2 Il flusso termina o il GM modifica lo stato della stanza (tramite altro UC).

4a. Guasto tecnico durante la partita:

4a.1 Un elemento della stanza si rompe impedendo il proseguimento del gioco.

4a.2 Il Game Master forza l'interruzione della sessione.

4a.3 Il Sistema chiede di specificare la motivazione, annulla la sessione corrente e transita la Stanza direttamente nello stato "In Manutenzione".

Tabella 2.3: Template del Caso d'Uso: Controllo Sessione di Gioco

Capitolo 3

Progettazione Architetturale e di Dominio

Il processo di transizione dall'analisi dei requisiti alla progettazione ha seguito i principi dell'Object-Oriented Design (OOD) e l'approccio BCE (Boundary-Control-Entity). In questo capitolo viene dettagliata la struttura statica del Domain Model, ponendo particolare enfasi sull'applicazione dei Design Pattern fondamentali per garantire la manutenibilità, l'estensibilità e il disaccoppiamento del sistema.

3.1 Domain Model (Class Diagram)

La Figura 3.1 illustra il diagramma delle classi relativo al nucleo logico dell'applicativo (Entity layer). Il modello non include volutamente i dettagli legati all'accesso ai dati (DAO) o all'interfaccia utente (View/CLI), concentrandosi esclusivamente sulle regole di business del franchise.

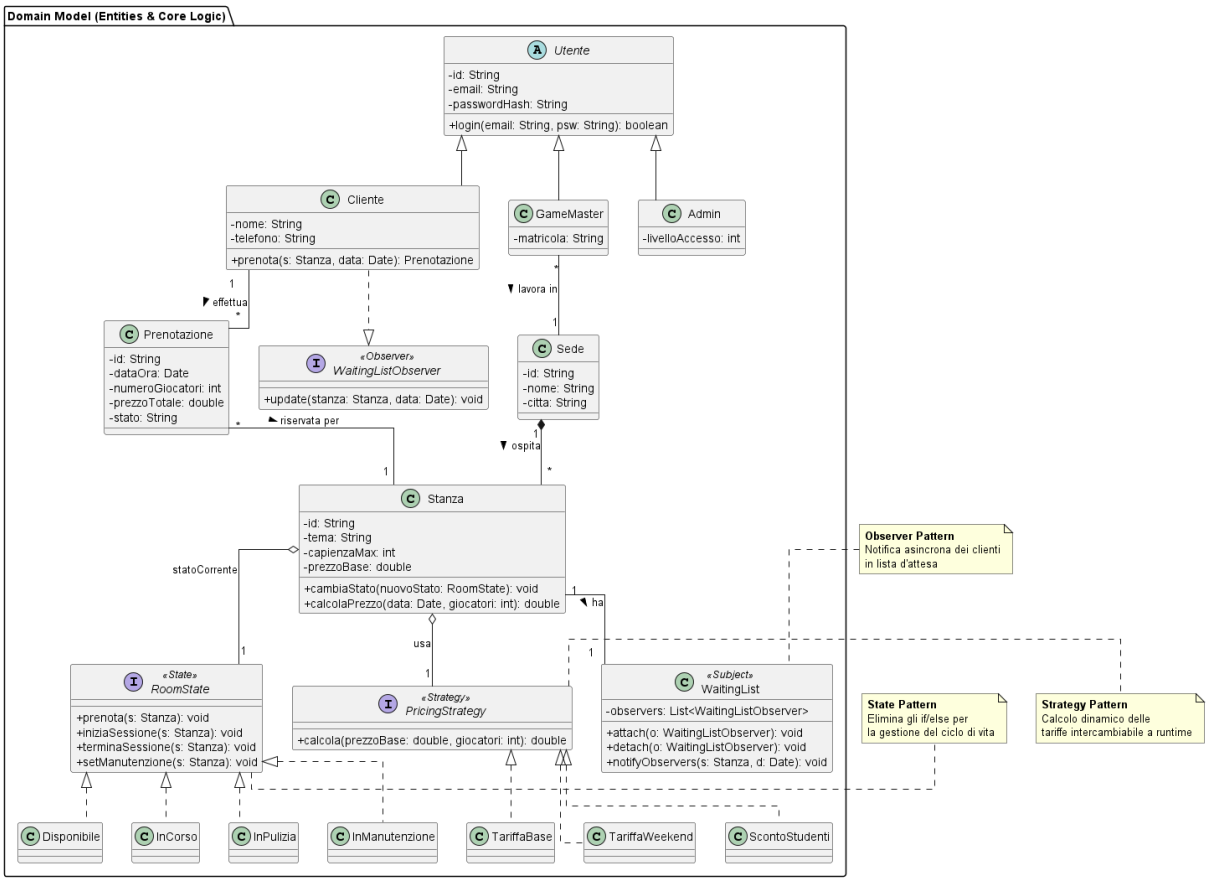

Figura 3.1: Diagramma delle Classi del Domain Model con Design Pattern

3.1.1 Scelte Architetturali e Design Pattern

L'architettura del dominio è stata concepita per risolvere nativamente le complessità emerse durante l'analisi dei casi d'uso (Capitolo 2), ricorrendo a tre Design Pattern della *Gang of Four (GoF)*:

- **State Pattern (Gestione ciclo di vita):** Come emerso nell'UC4, la `Stanza` attraversa fasi operative rigide (Disponibile, In Corso, In Pulizia, In Manutenzione). Invece di delegare il controllo a complessi blocchi `switch/case` basati su stringhe o enumerazioni, lo stato è stato reificato nell'interfaccia `RoomState`. Questo garantisce il rispetto del principio *Open-Closed*: l'aggiunta di un nuovo stato futuro non richiederà la modifica del codice della classe `Stanza`.

- **Strategy Pattern (Politiche di Prezzo):** Le regole di tariffazione dipendono da fattori esterni (giorno della settimana, festività, tipologia di clienti). La classe `Stanza` delega il calcolo del costo totale all'interfaccia `PricingStrategy`. L'amministratore (UC5) può così iniettare a runtime strategie diverse (es. `TariffaWeekend` o `ScontoStudenti`) variando l'algoritmo di calcolo in modo trasparente.

- **Observer Pattern (Lista d'Attesa asincrona):** Per soddisfare il requisito di notifica asincrona dell'UC3 (Iscrizione Lista d'Attesa), è stato implementato un meccanismo di Publish-Subscribe. La classe `WaitingList` agisce da *Subject*, mantenendo una lista di `WaitingListObserver` (interfaccia implementata dal `Cliente`). Quando una

prenotazione viene cancellata, il subject invoca il metodo `update()`, disaccoppiando totalmente la logica di notifica dal flusso principale.

Oltre ai pattern, si sottolinea l'uso dell'ereditarietà per la gestione degli attori (`Utente` come classe astratta generalizzata) e l'impiego della composizione forte (`*-`) tra `Sede` e `Stanza`, indicando che il ciclo di vita logico di una stanza è strettamente legato alla filiale fisica che la ospita.

3.2 Progettazione del Database

Per garantire la persistenza dei dati e supportare l'infrastruttura di un sistema centralizzato, il Domain Model Object-Oriented è stato mappato su un database relazionale (PostgreSQL). La transizione dal modello concettuale allo schema logico ha richiesto l'applicazione di specifiche strategie di *Object-Relational Mapping (ORM)*.

3.2.1 Diagramma Entity-Relationship

La Figura 3.2 mostra il diagramma Entità-Relazione (ER) del database, illustrando le tabelle principali, le chiavi primarie (PK) e i vincoli di chiave esterna (FK).

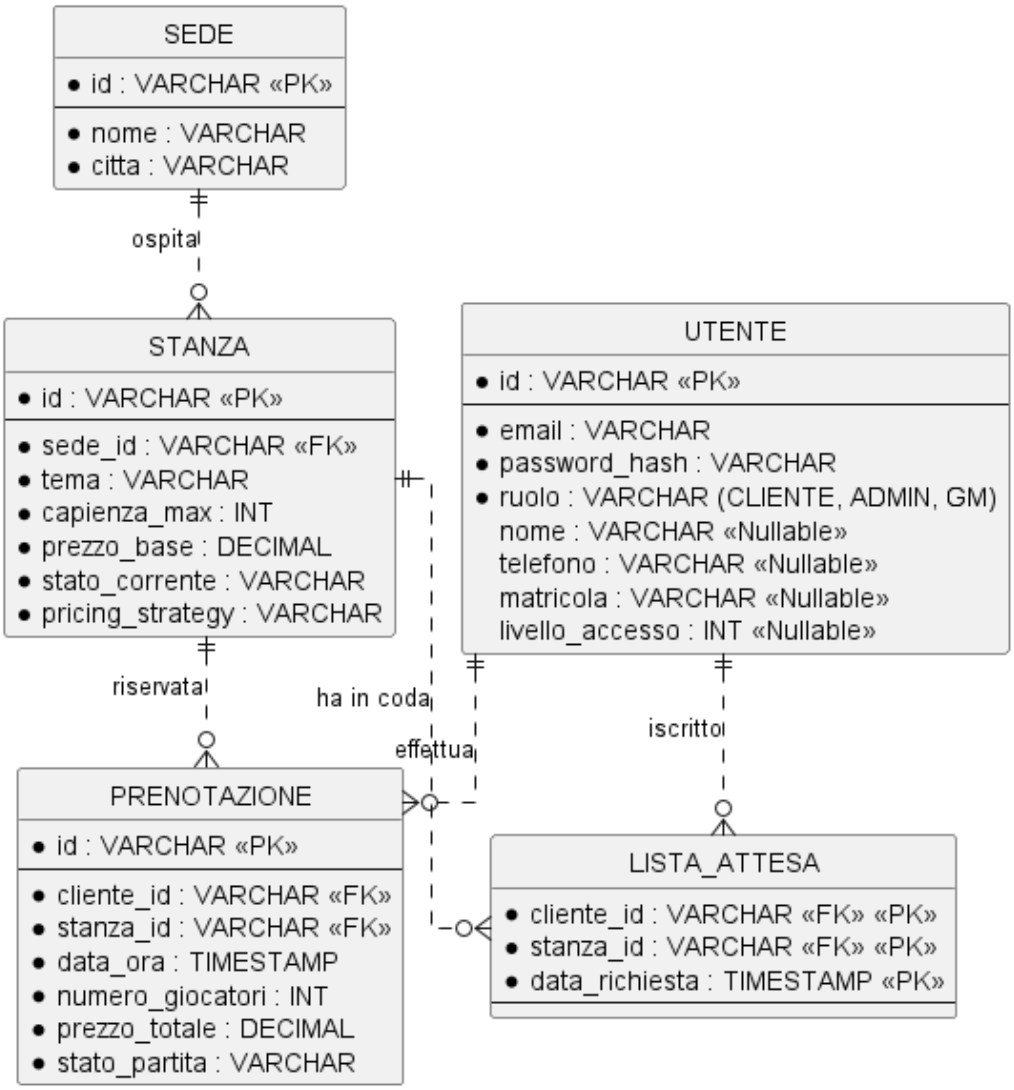

Figura 3.2: Diagramma Entity-Relationship (ER) del sistema

3.2.2 Strategie di Mapping e Modello Logico

Il design relazionale affronta la dicotomia tra il paradigma a oggetti e quello tabellare attraverso le seguenti scelte implementative:

- **Mapping dell'Ereditarietà (Single Table Strategy):** La gerarchia della classe astratta `Utente` (Cliente, GameMaster, Admin) è stata mappata utilizzando il pattern *Single Table Inheritance*. È stata creata un'unica tabella `UTENTE` provvista di una colonna discriminante (`ruolo`). Gli attributi specifici delle sottoclassi (es. `matricola` per il Game Master o `telefono` per il Cliente) sono stati resi *nullable*. Questa scelta ottimizza le query di autenticazione (UC1) evitando costose operazioni di `JOIN`.

- **Mapping dei Design Pattern (State e Strategy):** Le interfacce `RoomState` e `PricingStrategy` non diventano tabelle separate. All'interno della tabella `STANZA`, esse sono persistite tramite stringhe (es. `stato_corrente = 'IN_MANUTENZIONE'`). A livello di codice Java, i Data Access Object (DAO) utilizzeranno un meccanismo di *Factory* per istanziare l'oggetto concreto corrispondente al valore testuale letto dal database.

- **Associazioni Molti-a-Molti (Observer Pattern):** La logica Publish-Subscribe della Lista d'Attesa (UC3) si traduce relazionalmente nella tabella di *junction* `LISTA_ATTESA`. Questa tabella mappa la relazione N:N tra Clienti e Stanze, utilizzando una chiave primaria composita formata dalle foreign key e dal timestamp della richiesta.

Di seguito viene riportato lo **Schema Logico Relazionale** formalizzato:

- **SEDE**(id, nome, citta)
- **STANZA**(id, sede_id, tema, capienza_max, prezzo_base, stato_corrente, pricing_strategy)
- **UTENTE**(id, email, password_hash, ruolo, nome*, telefono*, matricola*, livello_accesso*)
  * = *attributo nullable a seconda del ruolo*
- **PRENOTAZIONE**(id, cliente_id, stanza_id, data_ora, numero_giocatori, prezzo_totale, stato_partita)
- **LISTA_ATTESA**(cliente_id, stanza_id, data_richiesta)

Capitolo 4

Implementazione e Architettura del Codice

Questo capitolo descrive la traduzione del Domain Model e dello Schema Logico in codice sorgente Java. L'architettura software è stata strutturata seguendo il pattern architetturale *Layered* (a strati), che ricalca logicamente il paradigma BCE (Boundary-Control-Entity), garantendo un elevato disaccoppiamento e facilitando il testing isolato dei componenti.

4.1 Architettura dei Package

L'applicazione è organizzata in quattro package principali, illustrati nel Package Diagram in Figura 4.1. Le dipendenze tra i package sono rigorosamente direzionali: lo strato di presentazione (`cli`) non accede mai direttamente al database, ma passa per i `controllers`, che a loro volta orchestrano il `domain` e si interfacciano con i `dao`.

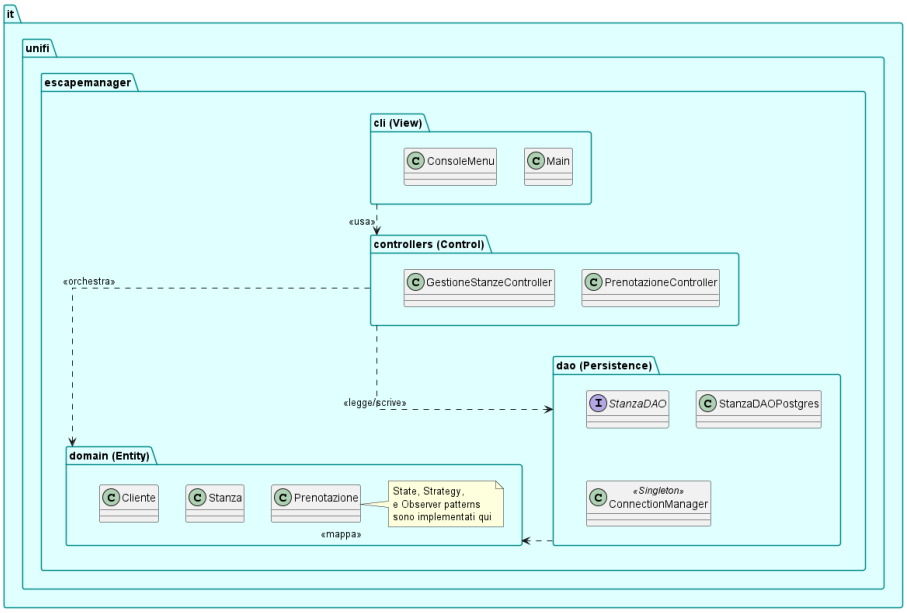

Figura 4.1: Package Diagram dell'architettura di sistema

4.2 Livello di Persistenza (Pattern DAO)

Per soddisfare i requisiti del modulo di Laboratorio di Informatica, l'accesso al database PostgreSQL è stato isolato tramite il pattern **Data Access Object (DAO)**. Questo garantisce che la logica di business rimanga del tutto agnostica rispetto alla tecnologia di persistenza.

La connessione al database è gestita tramite un **Singleton Pattern** (`ConnectionManager`), che assicura l'esistenza di un'unica istanza di connessione (o di un connection pool) condivisa per tutta l'applicazione, evitando l'overhead di connessioni multiple.

```java
public interface GenericDAO<T, ID> {
    void save(T entity) throws SQLException;
    T findById(ID id) throws SQLException;
    List<T> findAll() throws SQLException;
    void update(T entity) throws SQLException;
    void delete(ID id) throws SQLException;
}

public interface StanzaDAO extends GenericDAO<Stanza, String> {
    List<Stanza> findBySedeAndStato(String sedeId, String
        statoCorrente) throws SQLException;
}
```

Listing 4.1: Interfaccia DAO e uso dei Generics

4.3 Implementazione dei Design Pattern

Di seguito vengono riportati i frammenti di codice più significativi che mostrano l'implementazione pratica dei Design Pattern progettati nel Capitolo 3.

4.3.1 State Pattern

Lo State Pattern elimina la necessità di costrutti condizionali complessi per la gestione del ciclo di vita della `Stanza`. Ogni stato concreto implementa il comportamento specifico e gestisce, ove necessario, la transizione allo stato successivo.

```java
public interface RoomState {
    void prenota(Stanza stanza);
    void iniziaSessione(Stanza stanza);
    void terminaSessione(Stanza stanza);
    String getStatusString();
}

public class DisponibileState implements RoomState {
    @Override
    public void iniziaSessione(Stanza stanza) {
        // Logica di avvio...
        stanza.setStato(new InCorsoState()); // Transizione di stato
    }

    @Override
```

```java
    public void terminaSessione(Stanza stanza) {
        throw new IllegalStateException("Impossibile terminare una
            sessione non avviata.");
    }
    // ... altri metodi
}
```

Listing 4.2: Implementazione dello State Pattern

4.3.2 Strategy Pattern

Il calcolo del prezzo della prenotazione (UC2) è demandato alla `PricingStrategy`. Il Controller, in base alla data della prenotazione, inietta la strategia corretta all'interno dell'entità.

```java
public interface PricingStrategy {
    double calcola(double prezzoBase, int numeroGiocatori);
}

public class TariffaWeekend implements PricingStrategy {
    private static final double SOVRAPPREZZO = 1.20; // +20%

    @Override
    public double calcola(double prezzoBase, int numeroGiocatori) {
        return (prezzoBase * numeroGiocatori) * SOVRAPPREZZO;
    }
}
```

Listing 4.3: Implementazione dello Strategy Pattern

4.3.3 Observer Pattern

La lista d'attesa (UC3) implementa un sistema Publisher-Subscriber puro in Java, disaccoppiando l'evento di cancellazione di una prenotazione dall'invio effettivo delle notifiche.

```java
public interface WaitingListObserver {
    void update(Stanza stanza, LocalDateTime slotLiberato);
}

public class WaitingList {
    private List<WaitingListObserver> observers = new ArrayList<>();

    public void attach(WaitingListObserver observer) {
        observers.add(observer);
    }

    public void notifyObservers(Stanza stanza, LocalDateTime slot) {
        for (WaitingListObserver obs : observers) {
            obs.update(stanza, slot); // Notifica asincrona
        }
    }
}
```

Listing 4.4: Implementazione dell'Observer Pattern

Capitolo 5

Collaudo e Testing

La fase di verifica e validazione del sistema **EscapeManager** è stata condotta seguendo un approccio multilivello, utilizzando il framework **JUnit 5**. L'obiettivo del collaudo non è stato solo quello di garantire l'assenza di difetti evidenti (bug), ma soprattutto di dimostrare la robustezza dell'architettura e la perfetta aderenza ai requisiti funzionali delineati nel Capitolo 2.

5.1 Testing Strutturale (White-Box)

I test strutturali (o Unit Test) si sono concentrati sulle classi del `Domain Model`, con l'obiettivo di massimizzare la copertura (Code Coverage) dei rami condizionali e verificare la correttezza algoritmica, in totale isolamento dalle dipendenze esterne.

Particolare attenzione è stata dedicata al collaudo dei Design Pattern. Il Listing 5.1 mostra un test parametrizzato per la verifica delle politiche di prezzo dinamiche (*Strategy Pattern*).

```java
class PricingStrategyTest {

    @Test
    void testTariffaWeekendSovrapprezzo() {
        // Setup
        Stanza stanza = new Stanza("R01", "Horror Asylum", 6, 20.0);
        PricingStrategy weekendStrategy = new TariffaWeekend();
        stanza.setPricingStrategy(weekendStrategy);

        // Execution: 4 giocatori a 20 euro base = 80 euro. +20%
           weekend = 96.0 euro
        double prezzoCalcolato = stanza.calcolaPrezzo(4);

        // Assertion
        assertEquals(96.0, prezzoCalcolato, 0.01,
            "Il calcolo della tariffa weekend non applica
               correttamente il sovrapprezzo del 20%");
    }
}
```

Listing 5.1: Unit Test per la logica di calcolo del prezzo (Strategy Pattern)

5.2 Testing di Integrazione (DAO e Database)

Per verificare il corretto interfacciamento tra l'applicazione Java e il database PostgreSQL (livello ORM), sono stati implementati dei test di integrazione sulle classi DAO. Per garantire la **ripetibilità** e l'isolamento dei test (evitando che l'esecuzione di un test sporchi i dati del successivo), è stato sfruttato il costrutto `@BeforeEach` di JUnit per lanciare uno script SQL di *reset e re-inizializzazione* del database prima di ogni metodo di test.

```
class StanzaDAOTest {
    private StanzaDAO stanzaDAO;

    @BeforeEach
    void setUpDatabase() throws Exception {
        // Esegue reset.sql e default.sql per riportare il DB a uno
           stato noto
        DatabaseHelper.resetTestDatabase();
        stanzaDAO = new StanzaDAOPostgres();
    }

    @Test
    void testFindStanzeDisponibiliBySede() throws SQLException {
        // La sede "FI01" nel db di test ha 2 stanze DISPONIBILI e 1
           IN_MANUTENZIONE
        List<Stanza> disponibili = stanzaDAO.findBySedeAndStato("FI01
           ", "DISPONIBILE");

        assertNotNull(disponibili);
        assertEquals(2, disponibili.size());
        assertTrue(disponibili.stream().allMatch(s -> s.getStato().
           equals("DISPONIBILE")));
    }
}
```

Listing 5.2: Integration Test per StanzaDAO con reset dello stato

5.3 Testing Funzionale (Black-Box e Tracciabilità)

Il livello più alto di collaudo ha riguardato i `Controllers`. In questa fase (Black-Box Testing) la logica interna viene ignorata e il sistema viene testato esclusivamente in base agli input e agli output attesi.

Per garantire la massima **tracciabilità**, ogni test funzionale è stato mappato direttamente su uno specifico *Use Case Template* (Sezione 2.2). Sono stati collaudati sia i *Flussi Base* che i *Flussi Alternativi* (le eccezioni).

Il Listing 5.3 dimostra come il sistema gestisce fluidamente la *Race Condition* descritta nell'UC2 (Alternativa 3a), negando la prenotazione se lo slot risulta già occupato a database.

```
class PrenotazioneControllerTest {

    @Test
    void testPrenotazioneNegataPerSlotOccupato_UC2_Alt3a() {
```

```java
        // Setup: Il Cliente 2 tenta di prenotare uno slot appena occupato dal Cliente 1
        PrenotazioneController controller = new PrenotazioneController();
        String stanzaId = "R01";
        LocalDateTime slotRichiesto = LocalDateTime.of(2026, 10, 31, 21, 0); // Halloween, ore 21

        // Simulo la race condition pre-occupando lo slot
        controller.forzaOccupazioneSlot(stanzaId, slotRichiesto, "CLIENTE_01");

        // Execution & Assertion (Mi aspetto un'eccezione custom di business logic)
        SlotNonDisponibileException eccezione = assertThrows(
            SlotNonDisponibileException.class,
            () -> controller.effettuaPrenotazione("CLIENTE_02", stanzaId, slotRichiesto, 4)
        );

        assertEquals("Lo slot orario selezionato non e' piu' disponibile.", eccezione.getMessage());
    }
}
```

Listing 5.3: Functional Test tracciato sull'UC2 (Flusso Alternativo 3a)

5.4 Conclusioni

L'approccio metodologico adottato, guidato dai principi dell'Object-Oriented Design e supportato dall'uso estensivo dei Design Pattern, ha permesso di sviluppare **EscapeManager** come un sistema robusto, altamente coeso e disaccoppiato. La suite di collaudo ha confermato la solidità della logica di business e la corretta integrazione con il layer relazionale, soddisfacendo integralmente i requisiti posti in fase di analisi e fornendo un'architettura facilmente estensibile per sviluppi futuri (es. integrazione di sensori fisici nelle stanze tramite *Adapter Pattern* o implementazione di un'interfaccia grafica avanzata al posto dell'attuale CLI).